Task 6 Numerical Methods

CARL-EMIL ROHDE
KVB125@ALUMNI.KU.DK

ACM Reference Format:
Carl-Emil Rohde, kvb125@alumni.ku.dk. 2018. Task 6 Numerical Methods. In *Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 PROBLEM 1

1.1 6.1.1

b. Given the points $(x_0, y_0) = (7, 146)$, $(x_1, y_1) = (1, 2)$, and $(x_2, y_2) = (2, 1)$:
First, I find the divided differences $f[x_0, x_1]$, $f[x_1, x_2]$, and $f[x_0, x_1, x_2]$.

$$f[x_0, x_1] = \frac{y_1 - y_0}{x_1 - x_0} = \frac{2 - 146}{1 - 7} = \frac{-144}{-6} = 24$$
$$f[x_1, x_2] = \frac{y_2 - y_1}{x_2 - x_1} = \frac{1 - 2}{2 - 1} = \frac{-1}{1} = -1$$
$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{-1 - 24}{2 - 7} = \frac{-25}{-5} = 5$$

Now, the polynomial in Newton form is:

$$P(x) = y_0 + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1)$$
$$P(x) = 146 + 24(x - 7) + 5(x - 7)(x - 1)$$
$$P(x) = 146 + 24x - 168 + 5x^2 - 40x + 35$$
$$P(x) = 5x^2 - 16x + 13$$

So the polynomial of least degree that interpolates the given data points is $P(x) = 5x^2 - 16x + 13$
c. First i arrange the data in increasing order of x:

x	1	2	3	7
y	2	1	10	146

Then I Calculate the first, second, and third order forward differences of y:
First-order differences: - $\Delta y_0 = 1 - 2 = -1$ - $\Delta y_1 = 10 - 1 = 9$ - $\Delta y_2 = 146 - 10 = 136$
Second-order differences: - $\Delta^2 y_0 = 9 - (-1) = 10$ - $\Delta^2 y_1 = 136 - 9 = 127$
Third-order differences: - $\Delta^3 y_0 = 127 - 10 = 117$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Association for Computing Machinery.
Manuscript submitted to ACM

Carl-Emil Rohde
kvb125@alumni.ku.dk

Then i use these to form the Newton interpolating polynomial $P(x)$:

$$\begin{aligned} P(x) &= y_0 \\ &\quad + (x - x_0)\,\frac{\Delta y_0}{1!} \\ &\quad + (x - x_0)\,(x - x_1)\,\frac{\Delta^2 y_0}{2!} \\ &\quad + (x - x_0)\,(x - x_1)\,(x - x_2)\,\frac{\Delta^3 y_0}{3!} \end{aligned}$$

Then i substitute the calculated and given values:

$$\begin{aligned} P(x) &= 2 \\ &\quad + (x-1)(-1) \\ &\quad + (x-1)(x-2)\frac{10}{2} \\ &\quad + (x-1)(x-2)(x-3)\frac{117}{6} \\ &= 2 - (x-1) + 5(x-1)(x-2) + 19.5(x-1)(x-2)(x-3) \end{aligned}$$

This is the interpolating polynomial of least degree for the given data set, in Newton's form.

1.2 6.1.16

My pesudocode:

```
1. Function EvaluateSumAndProduct(d: Array of Float, n: Integer) -> Float
    1.1. If n <= 0 Then
            Return 0.0  // Return 0 if array is empty or negative size

    1.2. Initialize Float u = 0.0  // Variable to store the sum
    1.3. Initialize Float product = 1.0  // Variable to store the product for each i

    1.4. For i = 1 to n:
            1.4.1. product = product * d[i]  // Update the product
            1.4.2. u = u + product  // Add the updated product to the sum

    1.5. Return u  // Return the final sum
```

Explanation:

We initialize ' u ' to 0 and 'product' to 1. We then loop through i from 1 to n. Inside the loop, we multiply the existing 'product' by $d[i]$ to get the product for the current i. We add this to ' u '. This avoids the need for a nested loop, and we get each term in the sum by doing just one multiplication and one addition. Therefore, the time complexity is $O(n)$, which is more efficient than the naive $O\left(n^2\right)$ approach.

1.3 7.1.6

I'll find the Taylor series expansions for each term in the approximation formula:

1. $f(x+2h)$

$$f(x+2h) = f(x) + 2hf'(x) + \frac{(2h)^2}{2!}f''(x) + \frac{(2h)^3}{3!}f'''(x) + \frac{(2h)^4}{4!}f^{(4)}(x) + O(h^5)$$

2. $f(x+h)$

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) + O\left(h^5\right)$$

3. $f(x-h)$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2!}f''(x) - \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x) + O\left(h^5\right)$$

4. $f(x-2h)$

$$f(x-2h) = f(x) - 2hf'(x) + \frac{(2h)^2}{2!}f''(x) - \frac{(2h)^3}{3!}f'''(x) + \frac{(2h)^4}{4!}f^{(4)}(x) + O(h^5)$$

I then substitute these expansions into the original formula:

$$\begin{aligned}
f'(x) \approx & \frac{1}{12h}[-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)] \\
= & \frac{1}{12h}\left[-\left(f(x) + 2hf'(x) + \frac{4h^2}{2!}f''(x) + \frac{8h^3}{3!}f'''(x) + \frac{16h^4}{4!}f^{(4)}(x)\right)\right. \\
& + 8\left(f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x)\right) \\
& - 8\left(f(x) - hf'(x) + \frac{h^2}{2!}f''(x) - \frac{h^3}{3!}f'''(x) + \frac{h^4}{4!}f^{(4)}(x)\right) \\
& \left. + \left(f(x) - 2hf'(x) + \frac{4h^2}{2!}f''(x) - \frac{8h^3}{3!}f'''(x) + \frac{16h^4}{4!}f^{(4)}(x)\right)\right]
\end{aligned}$$

After substituting, we need to simplify the terms:

$$\begin{aligned}
f'(x) \approx & \frac{1}{12h}\left[-f(x) - 2hf'(x) - 2h^2f''(x) - \frac{4h^3}{3}f'''(x) - \frac{4h^4}{3}f^{(4)}(x)\right. \\
& + 8f(x) + 8hf'(x) + 4h^2f''(x) + \frac{8h^3}{3}f'''(x) + \frac{2h^4}{3}f^{(4)}(x) \\
& - 8f(x) + 8hf'(x) - 4h^2f''(x) + \frac{8h^3}{3}f'''(x) - \frac{2h^4}{3}f^{(4)}(x) \\
& \left. + f(x) - 2hf'(x) + 2h^2f''(x) - \frac{4h^3}{3}f'''(x) + \frac{4h^4}{3}f^{(4)}(x)\right] \\
= & \frac{1}{12h}\left[12hf'(x) + O\left(h^5\right)\right] \\
= & f'(x) + O\left(h^4\right)
\end{aligned}$$

The error term for the approximation $f'(x)$ is $O\left(h^4\right)$, which shows that the approximation is fourth-order accurate.

1.4 7.3.4

To verify that this equation is a correct Gaussian formula, I can check if it gives exact results for polynomials of degree up to $2n+1$, which in this case is 3 . Essentially, we are checking if the equation holds for $f(x) = 1, f(x) = x, f(x) = x^2$, and $f(x) = x^3$:

1. $f(x) = 1$

$$\int_{-1}^{1} 1dx = 2$$

$$f(-\alpha) + f(\alpha) = 1 + 1 = 2$$

Carl-Emil Rohde
kvb125@alumni.ku.dk

2. $f(x) = x$

$$\int_{-1}^{1} x dx = 0$$

$$f(-\alpha) + f(\alpha) = -\frac{1}{\sqrt{3}} + \frac{1}{\sqrt{3}} = 0$$

3. $f(x) = x^2$

$$\int_{-1}^{1} x^2 dx = \frac{2}{3}$$

$$f(-\alpha) + f(\alpha) = \left(-\frac{1}{\sqrt{3}}\right)^2 + \left(\frac{1}{\sqrt{3}}\right)^2 = \frac{2}{3}$$

4. $f(x) = x^3$

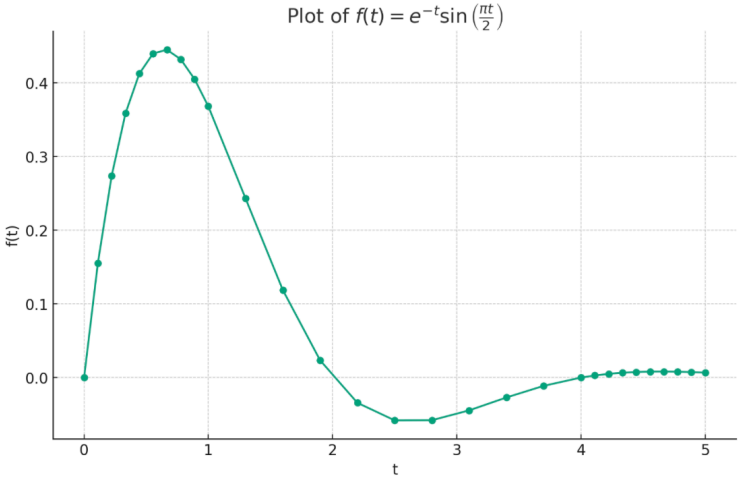
$$\int_{-1}^{1} x^3 dx = 0$$

$$f(-\alpha) + f(\alpha) = \left(-\frac{1}{\sqrt{3}}\right)^3 + \left(\frac{1}{\sqrt{3}}\right)^3 = 0$$

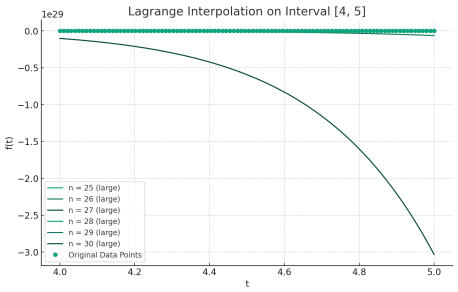
Since the formula holds for these polynomials, we can conclude that Equation (8) is a correct Gaussian formula for $n = 1$.

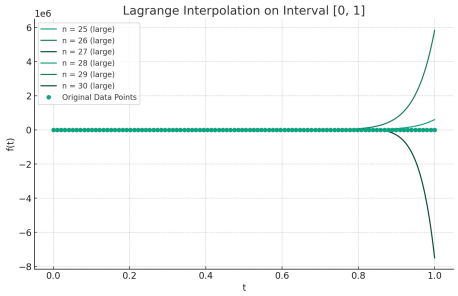
1.5 problem 2

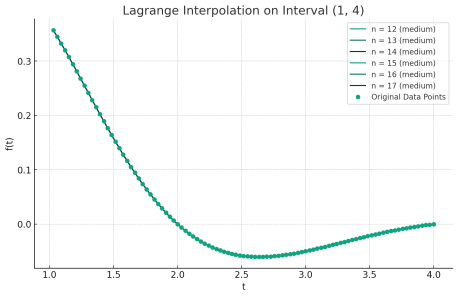
plotted the function $f(t) = e^{-t} \sin\left(\frac{\pi t}{2}\right)$ on the interval $[0, 5]$ using 30x-values:

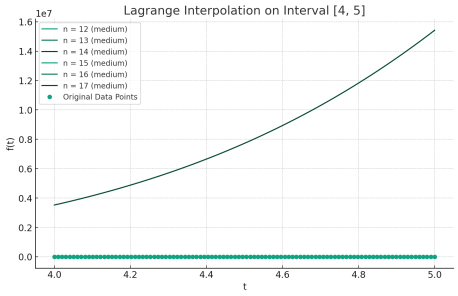


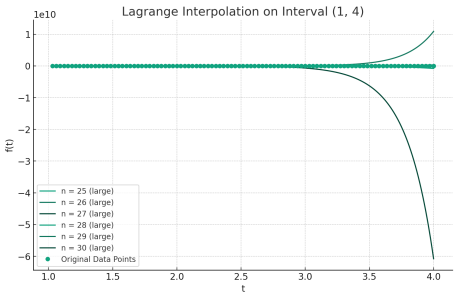
I've created Lagrange interpolations of the function $f(t) = e^{-t} \sin\left(\frac{\pi t}{2}\right)$ for different integer values of n on each interval: $[0, 1]$, $(1, 4)$, and $[4, 5]$. (Not sorted, sorry. Latex was messing)

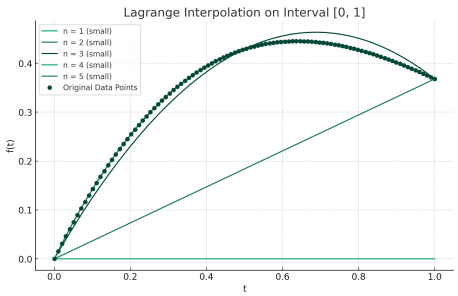

Lagrange Interpolation on Interval [4, 5]
1e29
n = 25 (large)
n = 26 (large)
n = 27 (large)
n = 28 (large)
n = 29 (large)
n = 30 (large)
Original Data Points
f(t)
t

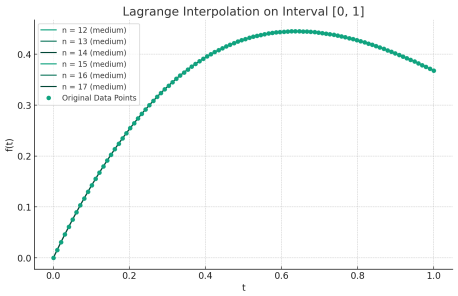

Lagrange Interpolation on Interval [0, 1]
1e6
n = 25 (large)
n = 26 (large)
n = 27 (large)
n = 28 (large)
n = 29 (large)
n = 30 (large)
Original Data Points
f(t)
t

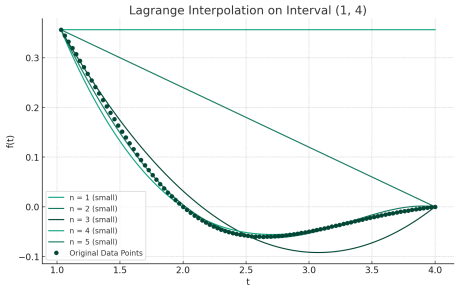

Lagrange Interpolation on Interval (1, 4)
n = 12 (medium)
n = 13 (medium)
n = 14 (medium)
n = 15 (medium)
n = 16 (medium)
n = 17 (medium)
Original Data Points
f(t)
t

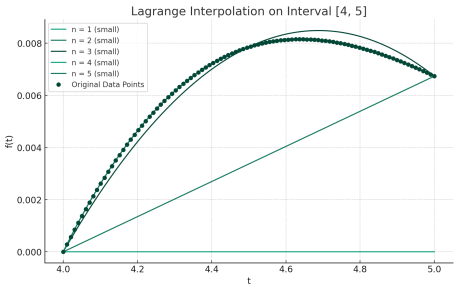

Lagrange Interpolation on Interval [4, 5]
1e7
n = 12 (medium)
n = 13 (medium)
n = 14 (medium)
n = 15 (medium)
n = 16 (medium)
n = 17 (medium)
Original Data Points
f(t)
t


Lagrange Interpolation on Interval (1, 4)
1e10
n = 25 (large)
n = 26 (large)
n = 27 (large)
n = 28 (large)
n = 29 (large)
n = 30 (large)
Original Data Points
f(t)
t


Lagrange Interpolation on Interval [0, 1]
n = 1 (small)
n = 2 (small)
n = 3 (small)
n = 4 (small)
n = 5 (small)
Original Data Points
f(t)
t

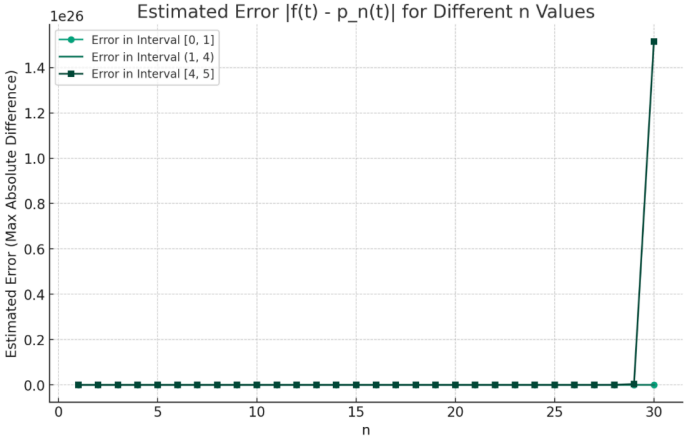

Lagrange Interpolation on Interval [0, 1]
n = 12 (medium)
n = 13 (medium)
n = 14 (medium)
n = 15 (medium)
n = 16 (medium)
n = 17 (medium)
Original Data Points
f(t)
t


Lagrange Interpolation on Interval (1, 4)
n = 1 (small)
n = 2 (small)
n = 3 (small)
n = 4 (small)
n = 5 (small)
Original Data Points
f(t)
t


Lagrange Interpolation on Interval [4, 5]
n = 1 (small)
n = 2 (small)
n = 3 (small)
n = 4 (small)
n = 5 (small)
Original Data Points
f(t)
t

Carl-Emil Rohde
kvb125@alumni.ku.dk

I've estimated the error $|f(t) - p_n(t)|$ for each natural number n from 1 to 30 , following the x-value spacing rule described earlier. The plot shows the maximum absolute difference (estimated error) between the actual function $f(t)$ and its Lagrange interpolation $p_n(t)$ for t in the interval $[0, 3]$:



2 PROBLEM 4